VisionFood QAI — AI-Powered Quality Inspection for Food & Beverage Manufacturing

Project Review 2: Implementation & Architecture

Shreyas Bairy K S – 1BM23CD058
Sushanth S – 1BM23CD063
Ullas N – 1BM23CD067

B.M.S. College of Engineering, Bengaluru–560019
(Autonomous Institute, Affiliated to VTU)

Department of Computer Science and Engineering (Data Science)

Guide: Prof. Nayana N Kumar

Project Phase 1 [AY: 2025–26]

Outline

1. Problem Background
2. VisionFood QAI
3. AI Models
4. Pipeline & Decision Logic
5. REMEDY Engine
6. Software Architecture
7. Testing & Training
8. Literature Survey
9. Research Gap

Problem Background

- Food and beverage manufacturing demands **100% visual inspection** before products leave the line — yet today this is done by humans standing at the conveyor belt.
- Three critical problems with manual inspection:
  1. **Fatigue:** After 2–3 hours, miss-rates climb to **15–20%** [1]
  2. **Speed:** High-speed lines run at **200+ products per minute** — no human can keep pace sustainably
  3. **Waste:** Every rejected product is binned even if the defect could be fixed in seconds
- Four defect categories directly impact consumer safety and brand reputation: fill levels, packaging integrity, label compliance, and surface contamination.
- Deep learning and computer vision now make **automated, real-time inspection at production scale** achievable [2, 3].

VisionFood QAI — System Overview

What We Propose to Build

An AI-powered quality inspection system that will photograph every product on a conveyor belt, detect packaging defects in real time, **determine whether each defect can be fixed**, and route the product to the correct action — automatically.

Proposed Software Stack:

- Two-stage AI inference pipeline
- REMEDY defect remediation engine
- REST API service
- Database layer
- Web analytics dashboard
- Message bus / event streaming
- Containerized deployment
- Automated test suite

Target: low-latency inference on edge hardware

The Four Defect Classes

1. Improper Filling

Too little or too much product inside. Detected via visible fill level, depth map, or thermal sensing.
Risk weight: 0.45

3. Label Misalignment

Crooked, wrinkled, missing, or mis-positioned labels. Brand compliance and barcode readability issue.
Risk weight: 0.30

2. Packaging Damage

Dents, cracks, tears, or seal failures. Product may be fine inside but the container is compromised.
Risk weight: 0.60

4. Surface Contamination

Stains, mould spots, foreign particles, or residue on packaging. **Highest food-safety risk.**
Risk weight: 0.90

Model 1 — YOLOv11 (Detection Stage)

What it is: Newest YOLO family model (Ultralytics, Oct 2024). Scans a full 640×640 image in a *single forward pass*, outputs bounding boxes with class labels and confidence scores.

Why we chose it:

- **Speed:** Designed for real-time inference on edge hardware
- **Efficiency:** Reported higher mAP than YOLOv8 with fewer parameters (Ultralytics, 2024)
- **Portability:** Native ONNX export; runs on any hardware without PyTorch

Role in pipeline: First-stage detector. If *no* defects found: **fast-exit PASS**. If detections exist: passes bounding boxes to the classification stage.

Planned training: Transfer learning from a COCO-pretrained YOLOv11n checkpoint. Fine-tuned with AdamW optimizer, cosine LR decay, mosaic and MixUp augmentation.
Target: mAP@50 $\geq$ 0.80.

Model 2 — EfficientViT-M5 (Classification Stage)

What it is: Vision Transformer using *multi-scale linear attention* instead of standard quadratic attention. From the `timm` library. M5 is the largest in its mobile-efficient series.

Why chosen over ResNet / EfficientNet:

- **Better accuracy** at similar parameter counts on ImageNet benchmarks
- Lightweight enough for edge hardware — minimal latency added after YOLO
- Produces **calibrated probability scores** per class, enabling uncertainty estimation

Role in pipeline: Receives a cropped patch of the bounding box region from YOLOv11 (with context padding). Two-stage design (detect then classify) gives higher accuracy than either model alone.

Planned training: Pretrained on a large-scale image dataset (ImageNet). Fine-tuned with **focal loss** (to focus on hard borderline cases) and label smoothing. Backbone progressively unfrozen during training.

Monte Carlo Dropout — Uncertainty Quantification

What it is: MC Dropout is *not a separate model* — it is a technique applied to EfficientViT. Dropout layers remain **active during inference**. The same image is run through the model **20 times**; each pass differs due to random neuron masking.

What the 20 passes produce:

- **Mean confidence** μ — the final prediction score
- **Standard deviation** σ — how much the 20 passes disagree
- **95% confidence interval:** $[\,\mu - 2\sigma,\ \mu + 2\sigma\,]$

Decision rule: If σ exceeds a calibrated threshold, the prediction is **uncertain** $\Rightarrow$ escalate to human review instead of acting automatically. The exact threshold will be set during model validation.

Key Design Intent

EfficientViT will be exported to ONNX with dropout layers kept active at inference, enabling genuine MC Dropout without requiring PyTorch at deployment.

Proposed Edge Inspection Pipeline

1. Product breaks IR sensor beam → camera trigger signal
2. Camera captures frame
3. Preprocess: resize / letterbox → normalise → convert to model input tensor
4. **YOLOv11** runs via ONNX Runtime; NMS filters overlapping detections
5. **No detections?** → PASS verdict issued. **Done.**
6. **Defects found:** crop patch from highest-confidence bounding box
7. Normalise crop → pass to EfficientViT classifier
8. **MC Dropout:** run EfficientViT multiple times; compute mean μ, std σ, confidence interval
9. Verdict logic applied → PASS / FAIL / ESCALATE / REVIEW
10. FAIL / ESCALATE → **REMEDY engine:** compute severity S1–S4; select corrective action

Target: low-latency end-to-end inference on CPU and GPU

Backend integration (database, message bus, dashboard) is detailed in the Software Architecture section.

Verdict System — Five Tiers

Verdict	Condition	Action
PASS	No defects detected	Product continues; pipeline ends (fast path)
FAIL (High)	High confidence, low uncertainty	→ REMEDY engine; automatic action
FAIL + Flag	Moderate confidence	→ REMEDY engine *and* human review queue
ESCALATE	Low confidence	→ Human inspector queue; no auto-action
REVIEW	Very low confidence	Logged for offline analysis only

Design principle: The system *never acts automatically on a low-confidence prediction* and *never wastes human time reviewing a prediction the model is already certain about.*

REMEDY Engine — Novel Contribution

What Makes VisionFood QAI Different

Most defect detection systems give two options: **Pass** or **Reject**. The REMEDY engine gives **five**, routing each FAIL to the most economical resolution instead of the bin.

Severity Score Formula (indicative weights, subject to calibration):

$$\text{score} = w_1 \cdot \frac{A_{\text{defect}}}{A_{\text{product}}} + w_2 \cdot (1 - c) + w_3 \cdot w_{\text{class}} + w_4 \cdot p_{\text{retry}}$$

Severity Grades:

- **S1** (score < 0.30) — Minor cosmetic; fully remediable on-line
- **S2** (0.30–0.55) — Moderate; remediable at a correction station
- **S3** (0.55–0.80) — Serious; generally rejected
- **S4** (≥ 0.80) — Critical food-safety risk; hard reject, no remediation attempted

Class risk weights (indicative): Label $<$ Fill $<$ Damage $<$ Contamination

REMEDY Engine — Remediation Actions

Action	Station	Triggers	Description
RELABEL	Station A	Label S1–S2	Automated applicator removes and re-applies the label correctly
REFILL	Station B	Fill S1–S2	Nozzle adjusts product volume to the correct fill level
REPACK / CLEAN	Station C	Damage S1, Contam S1	Cleaning or repackaging to restore packaging integrity
REJECT	Off-line	S3, S4, any class	Product diverted off line for investigation

Re-inspection loop: After any remediation the product re-enters the YOLOv11 pipeline. If remediation fails a defined number of times $\Rightarrow$ hard reject.

Expected Impact

A significant proportion of currently rejected products are expected to be recoverable via REMEDY, reducing material waste. Exact figures will be verified during evaluation.

Three-Tier Architecture

Edge Tier (GPU-capable device / edge hardware)

Runs the ONNX inference pipeline, REMEDY engine, and REST API server. **Fully offline** — no internet required.

Backend — Python REST API (async)

- Modular routers for inspection, analytics, reports, models, and live updates
- API authentication and request logging for traceability
- Metrics endpoint for monitoring request counts, latency, and verdict distribution

Deployment — Containerized (multiple services)

Inference worker · API server · Background task worker · Web dashboard · Experiment tracking · Reverse proxy

Database, Message Bus & Frontend

Database

- Supports dev (SQLite) and production (PostgreSQL) modes
- ORM-based async CRUD operations
- Tables for inspection records, detections, remediation actions, model versions, and reports
- Filtering by verdict, SKU, date range

Message Bus

- Event streaming for live inspection results
- WebSocket push to browser clients

Frontend (Web Dashboard)

Planned views:

1. **KPI Dashboard** — pass rate, defect rate, throughput
2. **Live Feed** — most recent result in real time
3. **History Table** — filterable inspection records
4. **Manual Inspect** — upload an image and see the result

Planned Test Strategy

Unit Tests

- Severity scorer: grade threshold boundaries
- Defect-class risk weight ordering
- Triage router: all class $\times$ grade routing combinations
- Config defaults and environment overrides

Integration Tests

- In-memory database with mocked model outputs
- API authentication and authorization
- Inspection CRUD and analytics endpoints

End-to-End Tests

- Full HTTP call chain: image upload $\rightarrow$ inference $\rightarrow$ DB save $\rightarrow$ analytics query
- No external server required

Design Goal

All test layers (unit, integration, e2e) will be in place before deployment, ensuring the system behaves correctly end-to-end.

Literature Survey: CNN & Transfer Learning

Theme	Author	Method	Dataset	Contribution	Insight
CNN Beverage QC	Chatchapol et al. [4]	CNN + Jetson NX	Beverage images	Real-time classification	Only fill-level; no multi-defect or localization support
Transfer Learning	Subramanian et al. [5]	ResNet50 + TL	MVTec AD	High accuracy via fine-tuning	ResNet50 too large for edge; no XAI or detection branch
Thermoforming	Banús et al. [1]	DenseNet, ResNet	Food packages	CNN comparison	Needs 10k+ images; weak generalization on small food datasets
Food Oil Quality	Sanjay et al. [6]	CNN classifier	Oil images	AI-based detection	Oil only; not extensible to packaged food defects
CV + DCAI	Xiong et al. [7]	End-to-end CV	Food packaging	Data-centric AI	Data quality outweighs model choice; informs our data strategy

Literature Survey: YOLO & Object Detection

Theme	Author	Method	Dataset	Contribution	Insight
YOLO Detection	Kavitha et al. [8]	YOLO + Arduino	Mfg.	Automated sorting	Arduino bottleneck limits throughput; not production-scale viable
YOLO + PLC	Nguyen et al. [9]	YOLOv8 + PLC	Industrial line	Vision + PLC integration	Proprietary PLC raises cost and limits transferability
YOLO Survey	Kavitha & Mamatha [10]	YOLOv5 review	Mfg.	YOLO comparison	YOLOv5 faster but less accurate on small or overlapping defects
Smart Mfg AI	Gediya [3]	CNN+YOLO pipeline	Smart factory	Complete QC pipeline	Depends on proprietary IoT hardware; difficult to reproduce
Food Pkg Defects	Liu et al. [11]	YOLOv5+CBAM	Food packaging	5-class defect detect.	Closest prior work; lacks Pass/Fail branch and uncertainty support

Literature Survey: IoT & Specialized Approaches

Theme	Author	Method	Dataset	Contribution	Insight
IoT + AI	Krishnan et al. [12]	CNN + IoT	Food samples	Image + sensor QC	Sensor calibration drifts over time; reduces reliability
RFID + AIoT	Cao et al. [13]	RFID + LSTM	Supply chain	Real-time traceability	Cloud round-trip adds high latency; not viable for high-speed lines
Multi-Sensor	Du [14]	ML + edge	Mfg.	Multi-sensor fusion	Fusion improves accuracy but raises system complexity and cost
Machine Vision	Farhangi et al. [15]	LabVIEW + edge	Bottle images	Pattern matching	Rule-based thresholds fail on new product variants
Anomaly Detect.	Truong & Luong [16]	Autoencoder	Food pkg.	Unsupervised learning	False-positive rate >20%; unreliable for production QC

Primary Base Paper & Literature Synthesis

Liu et al. (2025), PLOS ONE [11]

"Research on Mechanical Automatic Food Packaging Defect Detection Based on Improved YOLOv5"
Precision 0.96 / Recall 0.94 / F1 0.94 on a 7,400-image food packaging dataset.

What VisionFood QAI adds beyond this work:

1. **YOLOv11n** over YOLOv5: stronger accuracy with 22% fewer parameters
2. **EfficientViT-M5** classifier for calibrated, defect-aware Pass/Fail support
3. **MC Dropout** uncertainty with explicit 95% confidence intervals
4. **REMEDY engine**: 5 intelligent actions instead of 2 (pass/reject only)
5. **Full deployable stack**: REST API, live dashboard, Docker, audit trail

Key Trend: PRISMA review shows only **4%** of 150+ ML food-QC studies target beverages [2] — a major gap this project addresses.

Research Gap Identified

Key Research Gaps

1. **Limited food & beverage coverage:** Only 4% of ML-based food QC studies target beverages [2]
2. **Single-defect focus:** Most systems detect one defect type, not a unified multi-defect framework
3. **No remediation intelligence:** Nearest prior work [11] offers only pass/reject, wasting remediable products
4. **No uncertainty-aware decisions:** No published food-QC system combines localization, classification, and calibrated confidence intervals
5. **No end-to-end deployable system:** Few studies combine localization, classification, reporting, analytics, and a production-ready stack

How VisionFood QAI addresses this: A unified framework with YOLOv11n localization, EfficientViT-M5 classification, MC Dropout uncertainty, REMEDY remediation routing, and a fully deployable Docker-based software stack.

Project Objectives

General Objective: Build a software-based deep learning inspection system that detects, localizes, and remediates four visual defect types with measurable accuracy, uncertainty-aware decisions, and a production-ready software stack.

Specific Objectives:

1. Unified two-stage framework: YOLOv11n (detect) + EfficientViT-M5 (classify) *[planned]*
2. Transfer learning + augmentation for training on limited labeled data *[planned]*
3. MC Dropout uncertainty for PASS / FAIL / ESCALATE / REVIEW decisions *[planned]*
4. REMEDY engine for severity-graded defect triage and remediation routing *[planned]*
5. ONNX Runtime export for edge-efficient, PyTorch-free inference *[planned]*
6. Full software stack: REST API, database, dashboard, Docker, Prometheus *[planned]*
7. Evaluate against Top-1 accuracy, precision, recall, F1, mAP@0.5, latency *[pending dataset]*

Objectives 1–6 are planned and defined. Objective 7 will be validated once the dataset is acquired and models are trained.

Target Performance Benchmarks

Metric	Target	Justification
Top-1 Accuracy (EfficientViT-M5)	$\geq 95\%$	Quality acceptance threshold
Precision	$\geq 90\%$	Minimise false alarms and unnecessary remediation
Recall	$\geq 92\%$	Catch true defects consistently
F1-Score	$\geq 91\%$	Balanced classifier behaviour
mAP@0.5 (YOLOv11n)	$\geq 80\%$	Initial deployment-ready localisation
Escalation / Review Rate	5–15%	Conservative uncertainty management
Latency — CPU, ONNX	< 80 ms/frame	Real-time on edge hardware (from model benchmarks)
Latency — GPU	< 10 ms/frame	High-speed production line rate (from model benchmarks)
Throughput	≥ 12/sec	Compatible with pilot production rates
REMEDY Recovery Rate	TBD after evaluation	Of currently rejected units

Manual inspection baseline: 70–85% accuracy [1]

Design at a Glance

Proposed System

4	defect classes targeted
5	possible verdict outcomes
3	test layers (unit / integration / e2
3+	containerized deployment services

AI Pipeline

2-stage	detect then classify
***N*-pass**	MC Dropout (to be tune
4 actions	REMEDY routing outcor
Low latency	CPU deployment target
Low latency	GPU deployment target

Roadmap

Immediate
(Before Firm Pitch)

- Create 3 ONNX wrapper files in `inference/models/` (2 h)
- Annotate product images via Roboflow (2–3 days)
- Run training on Google Colab A100 (1–2 days)
- Live demo: webcam on product samples

Short Term
(Phase 2)

- HAFFN multi-modal fusion (RGB + thermal + depth)
- Jetson Orin NX with low-latency TensorRT INT8 inference
- CDAG-Net causal root-cause attribution
- Mamba-QC temporal defect forecasting

Long Term
(Production)

- Federated learning across factory sites
- Digital twin integration
- ERP/SAP via OPC-UA
- FDA 21 CFR Part 11 + ISO 22000 compliance portal

Dataset receipt $\rightarrow$ trained model deployed: 10 days

Research Paper

Overleaf Link

overleaf.com/read/ykvrykcycddr

References

N. Banús, I. Boada, P. Xiberta, P. Toldrà, and N. Bustins, "Deep learning for the quality control of thermoforming food packages," *Scientific Reports*, vol. 11, 2021.

K. G. Liakos, V. Athanasiadis, E. Bozinou, and S. I. Lalas, "Machine learning for quality control in the food industry: A review," *Foods*, vol. 14, 2025.

N. C. Gediya, "Smart manufacturing: Real-time quality control with ai and image recognition," *Conference Proceedings*, 2024.

B. Chatchapol, W. Autanan, and W. Anan, "Liquid filling quality control system for beverage industry using image processing and cnn," in *2024 24th International Conference on Control, Automation and Systems (ICCAS)*, pp. 1–6, 2024.

R. R. Subramanian, M. Dhamini, M. Srija, M. Vaishnavi, and M. Vidyadhari, "Revolutionizing manufacturing quality control with optimized resnet50: A deep learning approach for real-time defect detection," in *2025 International Conference on Computational Robotics, Testing and Engineering Evaluation (ICCRTEE)*, 2025.

S. Sanjay *et al.*, "Food oil quality and usage detection using ai," in *2024 10th International Conference on Communication and Signal Processing (ICCSP)*, pp. 1190–1194, 2024.

F. Xiong, N. Kühl, and M. Stauder, "Designing a computer-vision-based artifact for automated quality control: a case study in the food industry," *Annals of Operations Research*, 2024.

K. K. S, M. C. G, and R. T. V, "Deep learning based product defect detection for sustainable smart manufacturing," in *2024 5th IEEE Global Conference for Advancement in Technology (GCAT)*, pp. 1–6, 2024.

X.-T. Nguyen, T.-T. Mac, Q.-D. Nguyen, and H.-A. Bui, "An industrial system for inspecting product quality based on machine vision and deep learning," *Vietnam Journal of Computer Science*, vol. 12, no. 2, pp. 193–208, 2025.

K. K. S and M. C. G, "Automated product defect detection using image processing techniques for effective sorting and quality assurance: A survey," *International Journal of Innovative Science and Research Technology (IJISRT)*, vol. 9, no. 6, 2024.

References (cont.)

G. Liu, S. Zhang, L. Wang, X. Li, and G. Li, "Research on mechanical automatic food packaging defect detection model based on improved yolov5 algorithm," *PLOS ONE*, vol. 20, 2025.

P. Krishnan, U. S. Ebenezar, R. Ranitha, N. Purushotham, and T. S. Balakrishnan, "Ai-driven intelligent iot systems for real-time food quality monitoring and analysis," in *2024 International Conference on Trends in Quantum Computing and Emerging Business Technologies*, pp. 1–5, 2024.

Z. Cao, Z. Wu, and J. Gray, "Rfid tag-integrated multi-sensors with aiot cloud platform for food quality analysis," *Electronics*, vol. 15, no. 1, 2025.

T. Du, "Online measurement and detection technology and its optimized application in intelligent manufacturing environment," *Procedia Computer Science*, vol. 262, pp. 83–91, 2025.

O. Farhangi, E. Sheidaee, and A. Kisalaei, "Machine vision for detecting defects in liquid bottles: An industrial application for food and packaging sector," *Cluster Computing and Data Science*, 2024.

A. M. Truong and H. Q. Luong, "A non-destructive, autoencoder-based approach to detecting defects and contamination in reusable food packaging," *Current Research in Food Science*, vol. 8, p. 100758, 2024.

Thank You

Questions & Discussion